

NordicPump — Architecture

Alejandro Martínez • 2026-08-11 • v1.0

Documentación de arquitectura de NordicPump — comparador de precios de combustibles nórdicos

audience: juniors

NordicPump — Architecture

NordicPump es una **PWA mobile-first** que compara precios de combustibles (Euro 95 y Diésel) entre los 5 países nórdicos: **Suecia, Dinamarca, Finlandia, Noruega e Islandia**. El frontend es **Angular 22** y el backend es un proxy/caché en **Litestar (Python 3.14)**. No hay base de datos: el caché es file-based con escrituras atómicas. La app soporta **7 idiomas** y muestra precios en la moneda nativa de cada país.

Contexto

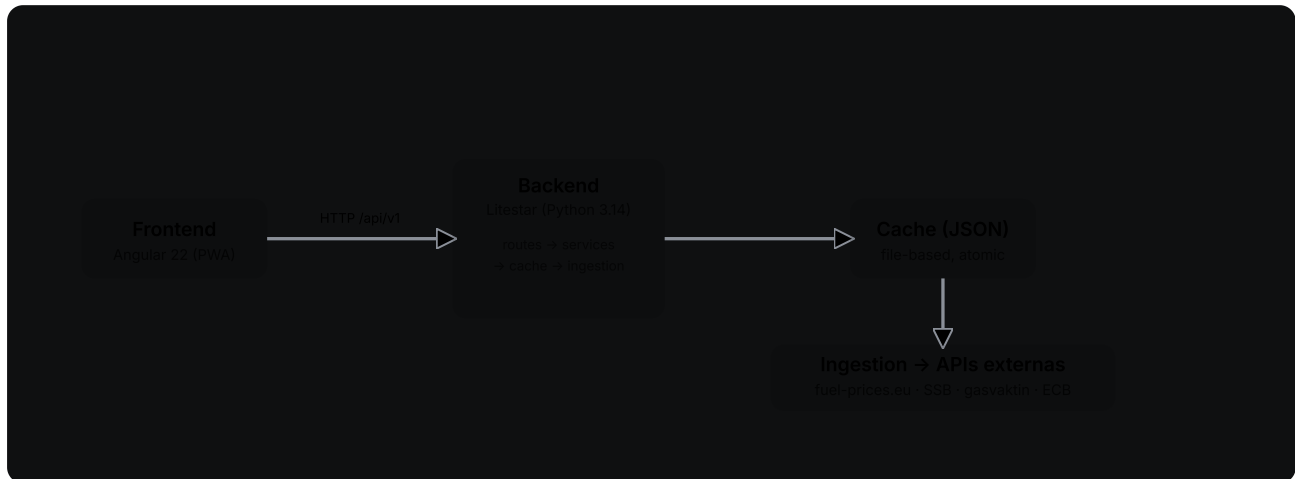
El problema: los precios de combustibles en los países nórdicos se publican en fuentes oficiales distintas (fuel-prices.eu para SE/DK/FI, SSB Statbank para NO, gasvaktin.is para IS), con cadencias distintas (semanal, mensual, quincenal) y en monedas distintas (SEK, DKK, NOK, EUR, ISK). Un conductor que cruza la frontera Suecia→Noruega no tiene forma fácil de comparar precios entre ambos países sin abrir 3 apps distintas y hacer conversiones mentales.

NordicPump resuelve eso: unifica las fuentes, normaliza a EUR internamente, y deja al usuario elegir en qué moneda ver los precios. Sin esta app, el usuario tendría que visitar 3-4 sitios oficiales, anotar precios, convertir monedas manualmente, y perder 10 minutos por consulta.

Why?

Elegimos construir NordicPump porque resuelve una necesidad real del día a día nórdico (comparar precios cruzando fronteras), nos obliga a integrar APIs públicas reales con sus particularidades (cadencias, formatos, monedas), y produce un proyecto de portafolio que le habla directamente al mercado técnico sueco: datos nórdicos, idiomas nórdicos, monedas nórdicas.

Arquitectura



Frontend

La interfaz que ve el usuario. Construida con **Angular 22** (standalone components, signals, change detection OnPush). Es una **PWA** con service worker para offline.

Responsabilidades del frontend:

- Renderizar el dashboard: KPI cards con precios actuales, calculadora de tanque, gráficos (historial, comparativa, desglose de impuestos)
- Manejar 7 idiomas (sv, da, nb, fi, en, es, is) con ngx-translate
- Convertir monedas en vivo (currency switcher: SEK/EUR/DKK/NOK/ISK)
- Funcionar offline con service worker + fallback page

Estructura de carpetas (frontend/src/app/):

- `core/` — servicios (CurrencyService, PriceApiService, ThemeService, CountryStateService, LanguageService) + guards
- `features/` — dashboard y about
- `shared/` — componentes reutilizables (charts, widgets, models, currency-switcher)
- `layout/` — header y footer
- `pwa/` — service worker bits (install prompt, freshness banner, offline fallback)

HTTP Contract

El puente entre frontend y backend. Está **documentado explícitamente** en `api-contract.yaml` porque el análisis estático no infiere llamadas HTTP — solo se ven los imports.

Endpoint	Frontend caller	Backend handler	Service
GET /api/v1/prices/{country}	PriceApiService.fetch(country)	prices.py:prices_endpoint	PriceService
GET /api/v1/rates	CurrencyService.loadRates()	rates.py:rates_endpoint	IngestionService
GET /health	(monitoring)	health.py:health_check	(purely monitoring)

Backend

El proxy/caché. Construido con **Litestar** (framework ASGI de Python, alternativa moderna a FastAPI). **Sin base de datos**: el caché vive en archivos JSON en disco.

Capas del backend (de afuera hacia adentro):

1. **Routes** (routes/) — reciben HTTP, validan el país, delegan al service. No tienen lógica de negocio.
2. **Services** (services/) — PriceQueryService (lee caché, cache-first), IngestionPipeline (escribe caché, orquesta ingestión).
3. **Cache** (cache/) — CacheStore (I/O atómica), CacheFreshness (lógica de tiempo).
4. **Ingestion** (ingestion/) — parsers de cada fuente externa (fuelpriceseu, ssb, iceland, ecb_rates).
5. **Scheduler** (scheduler.py + cadence/) — scheduler in-process async que dispara ingestión según cadencias.
6. **Models** (models/) — PriceRecord, Country (StrEnum), FuelType, CountryMeta.

Cache (file-based)

El corazón del backend. No usa SQLite ni Redis: usa **archivos JSON** en disco, con dos trucos clave:

1

Escrituras atómicas con `tempfile + os.replace` (POSIX rename). El archivo nunca queda corrupto a medias: se escribe en un archivo temporal y se renombra atómicamente. Si el proceso muere a la mitad, el archivo original sigue intacto.

2

Índices por país para $O(1)$ lookup. Además del archivo principal (`fuel-prices-eu.json` con todos los países), se escribe un índice por país (`fuel-prices-eu_idx_SE.json`, `_idx_DK.json`, etc.). Así `PriceQueryService` lee directamente el archivo del país sin escanear toda la lista.

Ingestion → APIs externas

Cada fuente externa tiene su propio parser y su propia cadencia:

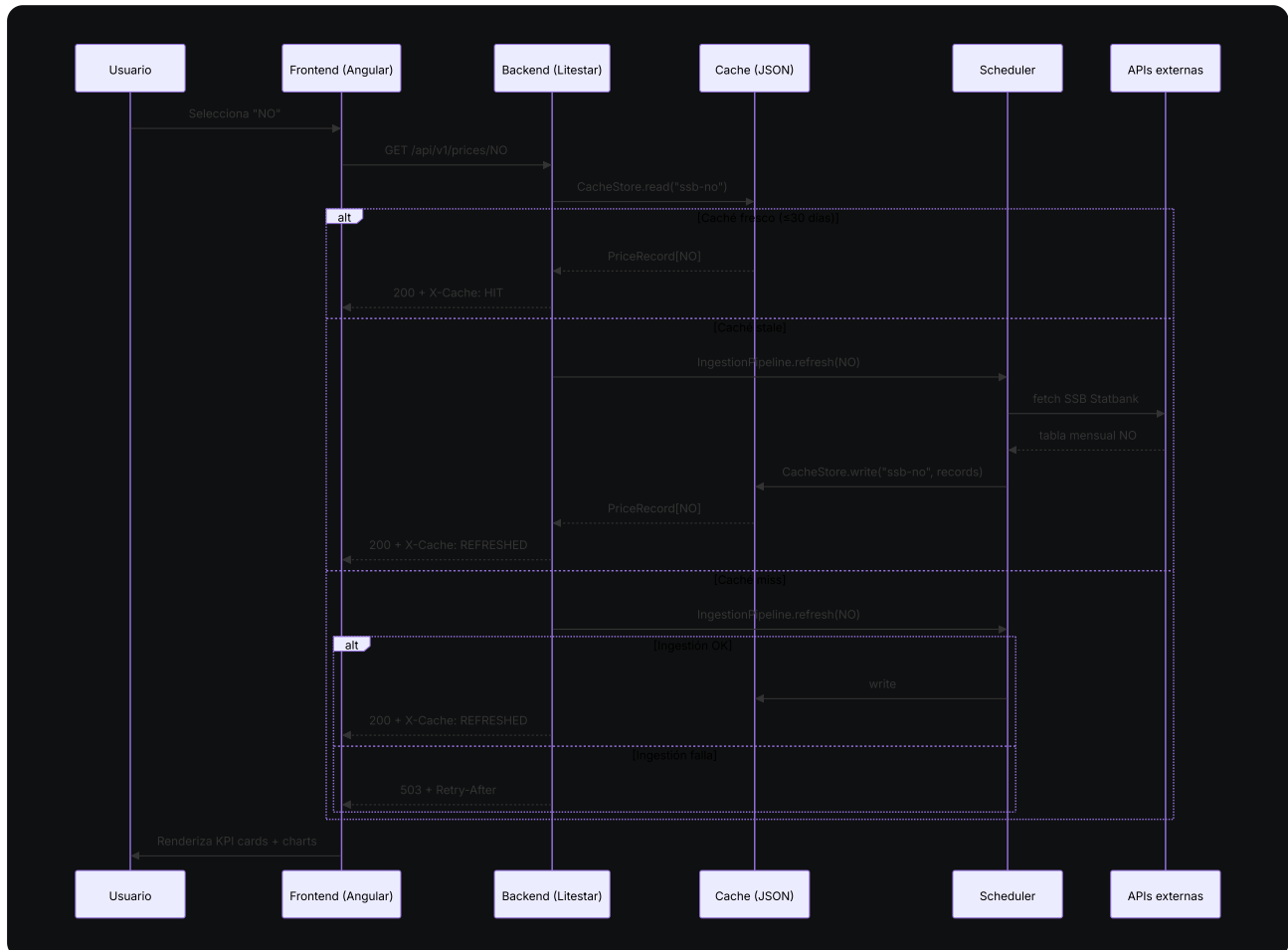
Fuente	Parser	Países	Cadencia	Por qué esa cadencia
<code>fuel-prices.eu</code>	<code>fuel_prices_eu.py</code>	SE, DK, FI	Domingos	Publican snapshot semanal los domingos
SSB Statbank	<code>ssb.py</code>	NO	1º del mes + día 15	Tabla mensual se publica a mediados de mes
<code>gasvaktin.is</code>	<code>iceland.py</code>	IS	Ventana 2 días	Estaciones actualizan cada 15 min, pero cachedeamos 2 días
ECB	<code>ecb_rates.py</code>	(rates)	Diario	Reference rates diarios del BCE

Warning

Islandia **no es UE**, así que el BCE (ECB) no publica EUR→ISK. `NordicPump` usa `open.er-api.com` como fuente para ISK, con fallback hardcodeado (`eur_isk_fallback: 140.00`) si esa API también cae.

Flujo de datos

¿Qué pasa cuando un usuario abre NordicPump y selecciona Noruega?



Tech Stack

Capa	Tecnología	Versión	Por qué
Frontend	Angular	22	Standalone components + signals son el presente de Angular. Stack demandado en mercado sueco (fintech, enterprise).
Backend	Litestar	latest	Framework ASGI con tipado estricto + OpenAPI. Más opinionated que FastAPI — te empuja a hacerlo bien.
Runtime backend	Python	3.14	StrEnum, performance, match statements.
Cache	JSON files	—	Sin DB. Atomic writes + per-country indices. Suficiente para un dataset de ~5 países × 2 combustibles.
Charts	Chart.js	4.x	Doughnuts, line charts. Tree-shakeable con registro manual de componentes.
i18n	ngx-translate	latest	Maduro, soporta 7 idiomas con interpolación <code>{{param}}</code> .
Tests frontend	Vitest	4.x	Rápido, jsdom, integration con Angular unit-test builder.
Tests backend	pytest	latest	Markers unit/integration, pytest-asyncio.
Tipado Python	mypy strict	—	<code>noPropertyAccessFromIndexSignature</code> fuerza acceso por <code>['key']</code> .
Linting Python	ruff	latest	Line-length 120, select E,F,I,N,W,UP.

Trade-offs (Decisiones clave)

Cada decisión técnica tuvo alternativas que se descartaron con razón. Estas son las más importantes:

Decisión	Elegido	Descartado	Razón
Framework backend	Litestar	FastAPI, Django	Litestar te empuja a tipado estricto + estructura limpia. FastAPI es muy libre (cada quien lo arma distinto). Django es overkill para un proxy/caché sin DB.
Persistencia	JSON files	SQLite, Redis	El dataset es chico (~10 registros por país, 5 países). SQLite agrega una dependencia y migraciones. Redis es infra externa. JSON + atomic rename es suficiente y portátil.
Scheduler	In-process async	cron, Celery	Una sola instancia, sin workers distribuidos. Async in-process evita dependencias externas (Redis broker para Celery) y funciona igual en dev y prod.
Enum país	StrEnum	Enum clásico	StrEnum serializa directamente a "SE" en JSON sin .value. Menos código, menos bugs.
Lazy charts	@defer (on viewport)	Lazy routes	Los charts son caros de renderizar (Chart.js). @defer los carga solo cuando el usuario scrollea a ellos. Gotcha: Playwright fullPage screenshot NO los dispara — hay que scrollear programáticamente en tests.
Source of truth país	countries.json + codegen	Types compartidos	Backend (Python) y frontend (TS) son runtimes distintos — no pueden compartir código. Un JSON fuente + script que genera enum Python + type TS elimina la deuda de "5 lugares que tocar por país nuevo".
Atomic write	POSIX rename	File lock	os.replace es atómico a nivel syscall en POSIX. Un lock (fcntl o file-based) agrega complejidad y edge cases (proceso muere con el lock tomado). Rename es más simple y garantiza consistencia.
Signals vs RxJS	Signals	RxJS	Signals son el futuro de Angular (reactividad simple). RxJS sigue para eventos complejos (HTTP, debouncing) pero para estado local, signals son más legibles y performantes.

Failure Modes (Modos de fallo)

Danger

Crítico: si el caché está vacío (miss) Y la ingestión falla (API externa caída), el backend devuelve **503 con Retry-After**. El usuario ve un mensaje de error traducido. No hay degradación elegante más allá de eso — no podemos inventar precios.

Warning

ECB caído: si el BCE no responde, los rates EUR→SEK/DKK/NOK usan fallback hardcodedo (11.50, 7.45, 12.00). Los precios convertidos pueden estar ligeramente desactualizados. Para ISK el fallback es 140.00 (vía open.er-api.com con su propio fallback).

Warning

Isla desconectada: el grafo de conocimiento del proyecto muestra 0 edges entre frontend y backend por defecto (análisis estático no infiere HTTP). Se soluciona con `api-contract.yaml` + `scripts/inject_contract_edges.py` que inyecta los edges manualmente.

Plan de escalabilidad

El sistema actual funciona para 5 países y un solo usuario concurrente. Si crece:

- 1 Más países** (ej. Países Bálticos): editar `countries.json` + correr python `scripts/generate_countries.py`. Las traducciones i18n y el flag SVG son manuales. El resto se autogenera.
- 2 Más concurrencia:** el file-based cache funciona mientras un solo proceso escribe. Para multi-proceso, migrar a SQLite (misma atomicidad, mejor concurrencia) o Redis. El refactor es pequeño porque `CacheStore` tiene solo 3 métodos públicos (`read`, `write`, `exists`).
- 3 Histórico largo:** actualmente el caché guarda solo el snapshot más reciente. Para series temporales largas, agregar append-only al archivo de caché (lista de snapshots) o tablas SQLite por país.

Glosario

Próximos pasos

- [] Deploy (Netlify frontend + Render/Railway backend)
- [] README profesional con screenshots y URL demo
- [] GitHub público
- [] Benchmark: cache hit vs miss (latencia)
- [] Más países bálticos (Estonia, Latvia, Lithuania) — solo editar `countries.json`

Info

Cómo se genera esta doc: `source.md` (este archivo) es la fuente única. Los pipelines `build-html.py`, `build-pdf.js`, `build-md.py`, y `build-narration.py` producen los formatos finales. Edita `source.md` y re-build para actualizar.